Python while loop

We use a while loop when we want to repeat a code block.

What is a while loop in Python?

In Python, the while loop statement repeatedly executes a code block while a particular condition is true.

```
count = 1
# condition: Run loop till count is less than 3
while count < 3:
    print(count)
    count = count + 1
```

In simple words, The while loop enables the Python program to **repeat a set of operations while a particular condition is true**. When the condition becomes false, execution comes out of the loop immediately, and the first statement after the while loop is executed.

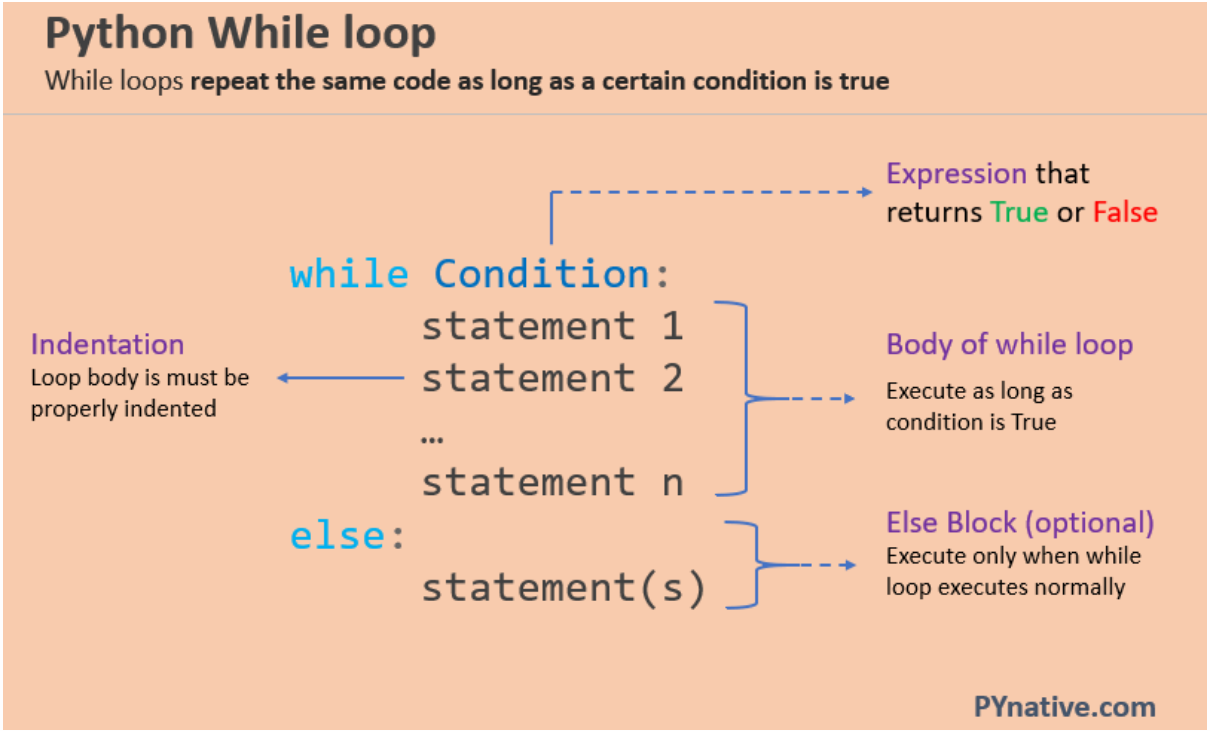
A while loop is a part of a control flow statement which helps you to understand the basics of Python.

We use a while loop when the number of iteration is not known beforehand. For example, if you want to ask a user to guess your luck number between 1 and 10, we don't know how many attempts the user may need to guess the correct number. In such cases, use a while loop.

- **Indefinite Iteration**: An unknown number of iterations. Ask the user to guess the lucky number. You don't know how many attempts the user will need to guess correctly. It can be 1, 20, or maybe indefinite. In such cases, use a `while` loop.
- **Finite Iterations**: Fixed number of iterations. Print the multiplication table of 2. In this case, you know how many iterations you need. Here you need 10 iterations. In such a case use for loop.

So, when number of iteration is not fixed always use the while loop.

AD



while loop in Python

Syntax for `while` loop

```
while condition:
      # Block of statement(s)
```

- The while statement checks the condition. The condition must return a boolean value. Either True or False.
- Next, If the condition evaluates to true, the while statement executes the statements present inside its block.
- The while statement continues checking the condition in each iteration and keeps executing its block until the condition becomes false.

Example of while loop in Python

Let's see the simple example to understand the while loop in Python

Example: Print numbers less than 5

In the above example, the while loop executes the body as long as the counter variable is less than 5. In each iteration, we are incrementing the counter by 1. Eventually, the counter variable will no longer be less than 5, and the while loop will stop executing.

```
count = 1
# run loop till count is less than 5
while count < 5:
    print(count)
    count = count + 1
```

Output:

```
1

2

3

4
```

Note: The loop with continuing forever if you forgot to increment counter in the above example

Example 2: Check how many times a given number can be divided by 3 before it is less than or equal to 10.

AD

In this example, the total iteration will vary depending on the number. when a number of iteration is not fixed always use the while loop.

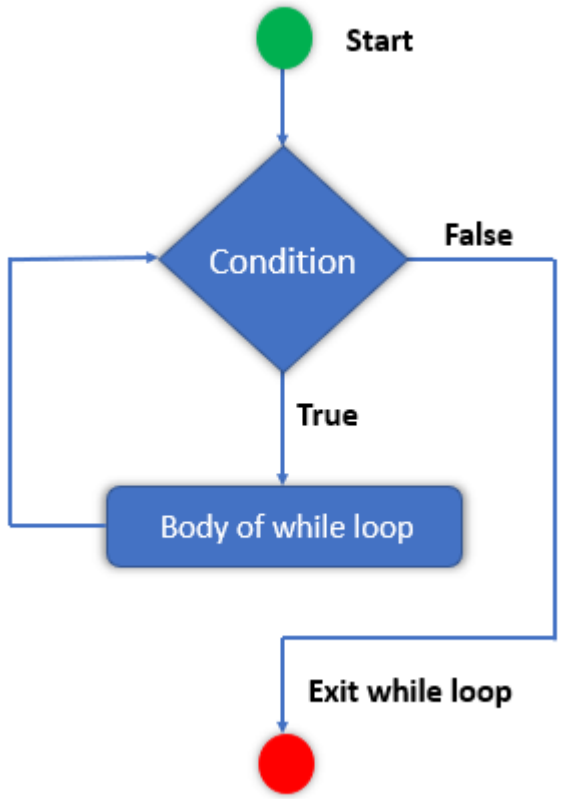
```
count = 0
number = 180
while number > 10:
    # divide number by 3
    number = number / 3
    # increase count
    count = count + 1
print('Total iteration required', count)
```

Output:

```
Total iteration required 3
```

Flowchart of while loop

AD



while loop flowchart

AD

Why and When to Use `while` Loop in Python

Now, the question might arise: when do we use a while loop, and why do we use it.

- **Automate and repeat tasks.**: As we know, while loops execute blocks of code over and over again until the condition is met it allows us to automate and repeat tasks in an efficient manner.
- **Indefinite Iteration**: The while loop will run as often as necessary to complete a particular task. When the user doesn't know the

number of iterations before execution, while loop is used instead of a for loop

- **Reduce complexity**: while loop is easy to write. using the loop, we don't need to write the statements again and again. Instead, we can write statements we wanted to execute again and again inside the body of the loop thus, reducing the complexity of the code
- **Infinite loop**: If the code inside the while loop doesn't modify the variables being tested in the loop condition, the loop will run forever.

Let's test the above statements.

Example 1: Assure proper input from user

In this example, we want a user to enter any number between 100 and 500. We will keep asking the user to enter a correct input until he/she enters the number within a given range.

```
number = int(input('Enter any number between 100 and 500 '))
# number greater than 100 and less than 500
while number < 100 or number > 500:
    print('Incorrect number, Please enter correct number:')
    number = int(input('Enter a Number between 100 and 500 '))
else:
    print("Given Number is correct", number)
```

D

Output:

```
Enter any number between 100 and 500 700

Incorrect number, Please enter correct number:

Enter a Number between 100 and 500 98

Incorrect number, Please enter correct number:

Enter a Number between 100 and 500 300

Given Number is correct 300
```

Example 2: Infinite while loop

```
# Infinite while loop
while True:
    print('Hello')
```

If-else in `while` loop

In Python, condition statements act depending on whether a given condition is true or false. You can execute different blocks of codes depending on the outcome of a condition. If-else statements always evaluate to either True or False.

We use the if-else statement in the loop when conditional iteration is needed. i.e., If the condition is True, then the statements inside the if block will execute othwerwise, the else block will execute.

Syntax of `if-else` statement

```
if condition :
     block of statements
else :
     block of statements
```

Let us see few operations with the help of examples.

Example: Print even and odd numbers between 1 to the entered number.

```
n = int(input('Please Enter Number '))
while n > 0:
    # check even and odd
    if n % 2 == 0:
        print(n, 'is a even number')
    else:
        print(n, 'is a odd number')
    # decrease number by 1 in each iteration
    n = n - 1
```

Output:

```
Please Enter Number 7
```

```
7 is a odd number

6 is a even number

5 is a odd number

4 is a even number

3 is a odd number

2 is a even number

1 is a odd number
```

Transfer statements in while loop

Loop control statements change the execution of the normal functioning of the loop. It is used when you want to exit a loop or skip a part of the loop based on the given condition. It also knows as transfer statements.

AD

There are three types of loop control statements break, continue and pass.

Break Statement

A break statement terminates the loop containing it. If the break statement is used inside a nested loop (loop inside another loop), it will terminate the innermost loop. Let us see the usage of the break statement with an example.

Example: Write a while loop to display each character from a string and if a character is number then stop the loop.

```
name = 'Jesaa29Roy'
size = len(name)
i = 0
# iterate loop till the last character
while i < size:
    # break loop if current character is number
    if name[i].isdecimal():
        break;
    # print current character
    print(name[i], end=' ')
    i = i + 1
```

Output:

```
J e s a a
```

Continue Statement

AD

continue is a statement that skips a block of code in the loop for the current iteration only. It doesn't terminate the loop but continues in the next iteration ignoring the loop's body after it. Let us see the use of the continue statement with an example.

Example: Write a while loop to display only alphabets from a string.

In this example, we will print only letters from a string by skipping all digits and special symbols

```
name = 'Jesaa29Roy'

size = len(name)
i = -1
# iterate loop till the last character
while i < size - 1:
    i = i + 1
    # skip while loop body if current character is not alphabet
    if not name[i].isalpha():
        continue
    # print current character
    print(name[i], end=' ')
```

Output:

```
J e s a a R o y
```

Pass Statement

Pass statement is a null statement. Nothing happens when the pass statement is executed. Primarily it is used in empty functions or classes. When the

interpreter finds a pass statement in the program, it returns no operation. Let us see the usage of the pass statement with an example.

AD

```
n = 4
while n > 0:
    n = n - 1
    pass
```

Nested `while` loops

In Python, while loop inside a while loop is known as nested loop.

In the nested while loop, the number of iterations will be equal to the number of iterations in the outer loop multiplied by the iterations in the inner loop. In each iteration of the outer loop inner loop execute all its iteration.

```
while expression:
    while expression:
        statemen(s) of inner loop
    statemen(s) of outer loop
```

AD

Example: Use nested while loop to print pattern

```
*

* *

* * *

* * * *
```

```
i = 1
# outer while loop
# 4 rows in pattern
while i < 5:
    j = 0
    # nested while loop
    while j < i:
        print('*', end=' ')
        j = j + 1
    # end of nested while loop
    # new line after each row
```

```
    print('')
    i = i + 1
```

AD

for loop inside a while loop

We can also use for loop inside a while loop as a nested loop. Let's see the same example using for loop inside the while loop.

```
i = 1
# outer while loop
while i < 5:
    # nested for loop
    for j in range(1, i + 1):
        print("*", end=" ")
    print('')
    i = i + 1
```

Else statement in `while` loop

In Python, we can use the else block in the while loop, which will be executed when the loop terminates normally. Defining the else block with a while loop is optional.

The else block will not execute in the following conditions:

- while loop terminates abruptly
- The break statement is used to break the loop

Example 1: Use while loop to print numbers from 1 to 5

```
i = 1
while i <= 5:
    print(i)
    i = i + 1
else:
    print("Done. while loop executed normally")
```

AD

Output:

```
1
```

```
2

3

4

5

Done. while loop executed normally
```

Example 2: Else block with break statement in a while loop.

In this case, else block will not be executed.

```
i = 1
while i <= 5:
    print(i)
    if i == 3:
        break
    i = i + 1
else:
    print("Done. while loop executed normally")
```

Output:

```
1

2

3
```

Reverse while loop

AD

A reverse loop means an iterating loop in the backward direction. A simple example includes:

- Display numbers from 10 to 1.
- Reverse a string or list

Example: Reverse a while loop to display numbers from 10 to 1

```
# reverse while loop
i = 10
while i >= 0:
    print(i, end=' ')
    i = i - 1
```

Output:

```
10 9 8 7 6 5 4 3 2 1 0
```

Iterate String using `while` loop

By looping through the *string* using while loop, we can do lots of string operations. Let us see some of the examples.

Example: while loop to iterate string letter by letter

```
name = "Jessa"
i = 0
res = len(name) - 1
while i <= res:
    print(name[i])
    i = i + 1
```

AD

Output:

```
J

e

s

s

a
```

Iterate a List using `while` loop

Python list is an ordered sequence of items. It is ordered by index numbers starting from 0. It is enclosed by square the '[]' brackets. Here are some of the examples of lists.

```
numbers = [1, 2, 4, 6, 7]
names = ["Messi", "Ronaldo", "Neymar"]
```

There are ways to iterate through elements in it. Here are some examples to help you understand better.

Example: Use while loop to iterate over a list.

```
numbers = [1, 2, 4, 5, 7]
size = len(numbers)
i = 0
while i < size:
    print(numbers[i])
    i = i + 1
```

Output:

AD

```
1
2
4
5
7
```